

UNIT 7 · WEEKS 10–12

Threads and concurrency

📅 Weeks 10–12 · dates to be confirmed </> Lab TD6 (solved + AI challenge) ~ 5 animations

MANUAL M7 · THEORY OF THE UNIT

Threads and concurrency

Core reading for unit 7 (≈ 2 h). The outputs are real: TD6 programs run on the course machine (28 x86-64 cores).

CONTENTS

§1	Threads	§4	Mutexes, semaphores and condition variables
§2	Race conditions and atomicity		
§3	Mutual exclusion: from software to hardware	§5	Deadlocks

§1 Threads

A **thread** is a sequential execution inside a process. The threads of a process share the whole address space —code, global data, heap— and the descriptors; each one has its own **stack**, its **registers** and its scheduling information. There is no memory protection between them: one thread's bug can corrupt everyone's data, and a signal like SIGSEGV ends the whole process. In Linux, the kernel schedules each thread as just another task (they are created with `clone`, sharing memory and descriptors).

The POSIX API: `pthread_create(&t, attr, f, arg)` runs `f(arg)` in a new thread; `pthread_exit` (or returning from `f`) ends it; `pthread_join` waits for its end, collects the return value and frees its resources (or `pthread_detach`, if nobody will wait for it). These functions return 0 or an error number, not -1. Compile with `-pthread`. Creating a thread is much cheaper than creating a process because no page tables are copied:

Real output

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

Real output

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5, local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2, local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3, local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6, local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7, local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8, local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

Practice this → A7.4 threads versus processes · TD6 · 1

§2 Race conditions and atomicity

A line of C like `x = x + 2` is several processor steps: reading `x` into a register, adding and writing. If two threads run lines like that on the same variable, the scheduler may interleave their steps in any way, and in some orders one write **overwrites** the other: that is a **race condition**. For `x = x+2 || x = x*3` with `x = 1` there are 20 possible interleavings; only the two serial ones give 9 or 5, the rest lose an update. In practice the "bad" orders are infrequent, which makes these bugs hard to reproduce: in TD6, eight threads selling 100,000 tickets without protection sell a few thousand too many.

An operation is **atomic** if it runs entirely or not at all, without another thread seeing an intermediate state. Since C11, two accesses to the same variable from different threads, without synchronization and with at least one write, are a *data race* and the behaviour is undefined; `volatile` does not fix it. For atomic operations, use `<stdatomic.h>` or `__atomic_*`.

Real output

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

Practice this → A7.1 race condition · TD6 · 2

§3 Mutual exclusion: from software to hardware

A **critical section** is a fragment that only one thread may run at a time (**mutual exclusion**). The "too much milk" problem shows how hard it is to achieve with reads and writes alone: looking and buying leaves a window; a shared note too (looking at it and putting it up are two steps); one

note per thread avoids buying twice but allows nobody to buy. **Peterson's algorithm** (own note + giving away the turn + waiting while the other has a note and it is their turn) is correct if memory is *sequentially consistent*.

Real processors are not. On x86, each core leaves its writes in a **buffer** and may read memory before they are visible to the others (the TSO model); ARM reorders even more. So the two Peterson threads may read each other's note as 0 and enter together. Measured on the course machine, with and without a memory fence (`mfence`):

Real output

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

That is why mutual exclusion is built with **hardware atomic instructions**: test-and-set, compare-and-swap (`lock cmpxchg` on x86), fetch-and-add. A lock that waits in a loop (*spinlock*) burns CPU and may suffer *priority inversion*; Linux's solution is the **futex**: if the lock is free it is taken with an atomic instruction in user space, and only if waiting is needed is the kernel called to sleep. Without contention, a mutex makes practically no system calls:

Real output

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex
./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00 0.000000 0 1 futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00 0.223730 27 8013 3116 futex
```

Practice this → A7.5 the milk problem · TD6 · 5

§4 Mutexes, semaphores and condition variables

A **mutex** (`pthread_mutex_lock` / `unlock`) protects a critical section: whoever finds the mutex taken sleeps until it is released. A **semaphore** (Dijkstra) is an integer ≥ 0 with two atomic operations: `sem_wait` (P: waits until positive and subtracts 1) and `sem_post` (V: adds 1 and wakes a waiting

thread). Initialized to 1 it is a lock; to 0, a notification between threads; to k, a limit of k users at a time. `sem_init` creates an anonymous one (shareable between processes with `pshared = 1` in shared memory) and `sem_open` a named one (it returns `SEM_FAILED` on failure).

The **producer-consumer** combines both: a buffer of `BUF` cells, an `empty` semaphore (free cells, starting at `BUF`), a `full` one (occupied cells, starting at 0) and a mutex for the shared indices. The producer waits if the buffer is full and the consumer if it is empty; nothing is lost or repeated:

Real output

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

To wait for an arbitrary condition, **condition variables** are used: with the mutex held, `while (!condition) pthread_cond_wait(&c, &m);`. `cond_wait` releases the mutex and sleeps atomically, and takes it again on waking up; whoever changes the condition calls `pthread_cond_signal` or `_broadcast`. `while` is used because there may be spurious wakeups.

Practice this → A7.2 producer-consumer · TD6 · 3

§5 Deadlocks

A **deadlock** is a set of threads waiting for each other forever. It appears only if **all four conditions** (Coffman) hold: mutual exclusion, holding resources while waiting for others, no preemption and circular wait. Since it depends on the execution order, it may not show up in thousands of tests. The dining philosophers show it: if each one takes the right chopstick first and then the left one, all may end up holding one and waiting for their neighbour's. Breaking one condition is enough: a **global order** of acquisition (the TD6 fix), limiting how many compete (a "waiter"), releasing and retrying (`trylock`), or detecting cycles in the allocation graph and recovering. In the Linux kernel, `lockdep` checks that locks are always taken in the same order.

Real output

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status: ${PIPESTATUS[0]}"
(124 = still blocked after 3 s)
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

Real output

```

$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals

```

Practice this → A7.3 the philosophers · TD6 · 4